AKADEMIA HANDLOWA NAUK STOSOWANYCH W RADOMIU — RADOM ACADEMY OF ECONOMICS

INSTRUKCJA LABORATORYJNA NUMER 1	
Przedmiot: Programowanie Niskopoziomowe	Tok: 2023/2024
Cel praktyczny zajęć: Zastosowanie podstawowych poleceń assemblera.	

1. Kalkulator liczb całkowitych

Korzystając z poniższych przykładów napisz program w języku assembler, który wykonuje każdą z podstawowych operacji arytmetycznych (dodawanie, odejmowanie, mnożenie, dzielenie), a następnie wyświetla wynik tej operacji na standardowym wyjściu w konsoli.

C: > Users > bartl > Desktop > sample.asm

```
section .text
global _start

_start:
    ; Dodawanie dwóch wartości
    mov rax, 10        ; Załaduj pierwszy argument (10) do rax
    add rax, 20        ; Dodaj drugi argument (20) do zawartości rax
```

C: > Users > bartl > Desktop > sample.asm

```
section .text
global _start

_start:
    ; Mnożenie 10 * 5
    mov rax, 10        ; Załaduj pierwszy argument (10) do rax
    mov rbx, 5         ; Załaduj drugi argument (5) do rbx
    mul rbx            ; Pomnóż zawartość rax przez zawartość rbx
```

Operacja	Polecenie	Przykład
Dodawanie	ADD	add dest, src
Odejmowanie	SUB	sub dest, src
Mnożenie	MUL	mul src
Dzielenie	DIV	div src
Inkrementacja	INC	inc dest
Dekrementacja	DEC	dec dest

Przykładowy wynik działania programu powinien być następujący:

```
ahnsuser@Ubuntu-AHNS-Prog-Niskopoziomowe:~/Desktop/assembly/prog_niskopoziomowe/imie_nazwisko_DD_MM_YYYY/lab1c1_pkt3$ ./hello
Pierwsza liczba: 100
Druga liczba: 5
Wynik dodawania: 105
ahnsuser@Ubuntu-AHNS-Prog-Niskopoziomowe:~/Desktop/assembly/prog_niskopoziomowe/imie_nazwisko_DD_MM_YYYY/lab1c1_pkt3$
```

W pierwszej wersji nie skupiaj się na pobraniu wartości od użytkownika, wykonaj każdą z operacji wyświetlając argumenty operacji oraz jej wynik, postaraj się w jednym skrypcie umieścić wszystkie działania. Zastanów się czy możliwe jest zastosowanie funkcji dla operacji, które są powtarzalne.

2. Pobranie wartości od użytkownika

Zapisz swój poprzedni program i załóż nowy katalog wraz z plikiem .asm oraz makefile. Tym razem ponownie wykonaj kalkulator zaczynając od prostego dodawania, pobierz argumenty od użytkownika za pomocą konsoli. Ściągawkę z pobieraniem wartości od użytkownika znajdziesz tutaj: https://github.com/kicbar/ahns-assembly/blob/master/src/lab1/hint_pkt6/input.asm

3. Porównywanie wartości podanych przez użytkownika.

Zapisz swój poprzedni program i załóż nowy katalog wraz z plikiem .asm oraz makefile. Napisz program, który przyjmuje od użytkownika dwie wartości całkowite a następnie porównuje je, wyświetlając informację o tym która z nich jest większa, skorzystaj z polecenia cmp oraz instrukcji skoku. Możesz zacząć od prostego kodu bez konieczności pobierania zmiennych od użytkownika.

C: > Users > bartl > Desktop > sample.asm

```
section .text
global _start

_start:
    mov eax, 5      ; Załaduj wartość 5 do rejestru eax
    mov ebx, 10     ; Załaduj wartość 10 do rejestru ebx

    cmp eax, ebx    ; Porównaj wartości w eax i ebx
    jz equal        ; Skocz do etykiety equal, jeśli wartości są równe
    jg greater      ; Skocz do etykiety greater, jeśli wartość w eax jest większa niż w ebx
    jl lesser       ; Skocz do etykiety lesser, jeśli wartość w eax jest mniejsza niż w ebx

equal:
    ; Kod do wykonania, jeśli wartości w eax i ebx są równe

greater:
    ; Kod do wykonania, jeśli wartość w eax jest większa niż w ebx

lesser:
    ; Kod do wykonania, jeśli wartość w eax jest mniejsza niż w ebx

```

4. Rozszerzenie operacji w kalkulatorze

W kalkulatorze z punktu 2, dodaj obsługę pozostałych operacji uwzględniając pobranie wartości od użytkownika.

5. Wybór operacji przez użytkownika

Zapisz swój poprzedni program i załóż nowy katalog wraz z plikiem .asm oraz makefile. Daj użytkownikowi możliwość wyboru operacji w kalkulatorze pobierając od niego informację o tym jakie działanie chce wykonać. Poproś o podanie argumentów, a następnie zwróć poprawny wynik.

6. Data Display Debugger – graficzny debuger

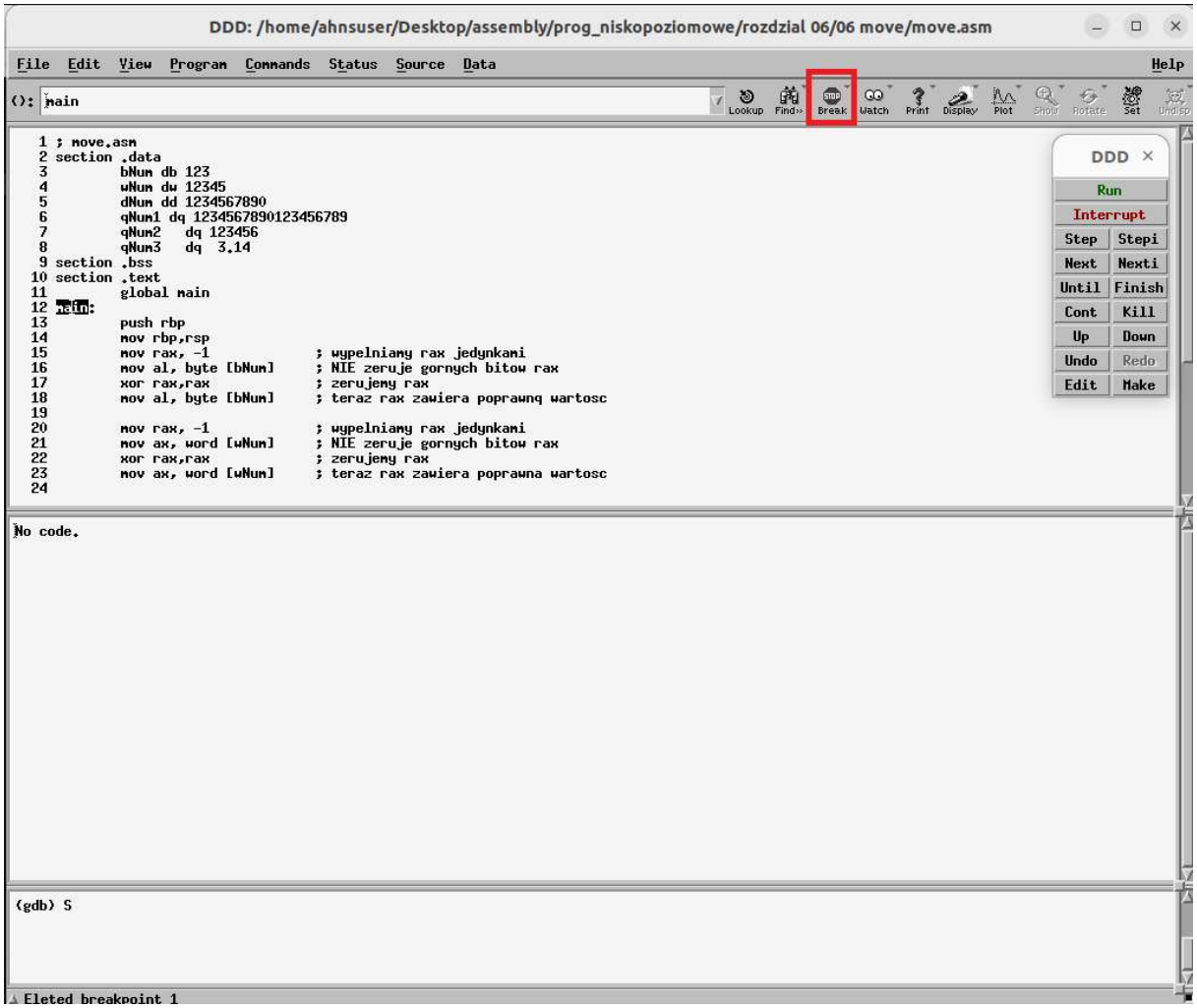
Zainstaluj debugger graficzny poleceniem: *sudo apt install ddd*

Przepisz poniższy kod źródłowy:

move.asm

```
; move.asm
section .data
        bNum db 123
        wNum dw 12345
        dNum dd 1234567890
        qNum1 dq 1234567890123456789
        qNum2   dq 123456
        qNum3   dq  3.14
section .bss
section .text
        global main
main:
        push rbp
        mov rbp,rsp
        mov rax, -1             ; wypelniamy rax jedynkami
        mov al, byte [bNum]     ; NIE zeruje gornych bitow rax
        xor rax,rax             ; zerujemy rax
        mov al, byte [bNum]     ; teraz rax zawiera poprawnq wartosc

        mov rax, -1             ; wypelniamy rax jedynkami
        mov ax, word [wNum]     ; NIE zeruje gornych bitow rax
        xor rax,rax             ; zerujemy rax
        mov ax, word [wNum]     ; teraz rax zawiera poprawna wartosc

        mov rax, -1             ; wypelniamy rax jedynkami
        mov eax, dword [dNum]   ; zeruje gorne bity rax

        mov rax, -1             ; wypelniamy rax jedynkami
        mov rax, qword [qNum1]  ; zeruje gorne bity rax

        mov qword [qNum2], rax  ; jednym z operandow jest zawsze rejestr
        mov rax, 123456         ; operandem zrodlowym moze byc wartosc bezposrednia

        movq xmm0, [qNum3]      ; specjalna instrukcja dla liczb zmiennoprzecinkowych
        mov rsp,rbp
        pop rbp
ret
```

makefile

```
move: move.o
    gcc -o move move.o -no-pie
move.o: move.asm
    nasm -f elf64 -g -F dwarf -l move.lst move.asm
```

Zbuduj program poleceniem make, a następnie uruchom program w debuggerze poleceniem: *ddd move*. Powinieneś zobaczyć nowe okno takie jak na poniższym zrzucie. Zaznacz fragment *main:* kodu w 12 linii, a następnie wybierz z panelu przycisk Break w celu ustawienia w tym miejscu brakpoint'a. DDD bazuje na GDB, dlatego też na dole znajduje się terminal, w którym możemy wprowadzać polecenia, alternatywą jest znajdujący się po prawej stronie graficzny panel wykonujący te same instrukcje. W dowolny sposób wykonaj polecenie Run, następnie z górnej zakładki Status wybierz pozycję Registers…, korzystając z polecenia step możesz teraz przechodzić do kolejnych breakpointów i obserwować zmieniające się rejestry.